HACK IN HILLS '26 • TRACK 04: BAYORA

AEGISBAY

Evidence-Gated AI Safety Testing & Blinded Zero-Trust Evaluation Enclave

Author & Team Lead	Ayush Sarkar (B.Tech CSE AI & ML, Brainware University)
Team Name	Awara Coders
Target Track	Track 04: Securing Adversarial AI Safety Testing Infrastructure
Security Classification	Production Zero-Trust Enclave Architecture
Cryptographic Root	RFC 8785 Canonical Merkle Tree + Ed25519 Signatures
Autonomous Sentinel	TinyFish Autonomous Web Sentinel (Verified Egress Contained)
Score Contamination	0.0% Contamination (Eliminated from 100.0% Baseline, $p < 0.001$)
Public Repository	https://github.com/Ayushnot41/AgeisBay
Date of Publication	October 22, 2026

EXECUTIVE MOTTO:
"You cannot benchmark AI safety on an unshielded server. When the testing arena itself leaks prompts, allows policy drift, and ignores timing side-channels, every safety metric is tainted. AegisBay turns the evaluation runtime into an immutable, mathematically verifiable cryptographic vault."

1. Executive Overview & The Problem We Solve

Current adversarial AI safety benchmarks evaluate model defenses on insecure, shared infrastructure where model providers (Blue Teams) can observe live attack prompts, leak KV-cache state across test sessions, tune defensive filters mid-evaluation, or infer test semantics via microarchitectural timing side-channels.

When an AI benchmark reports a "95% defense rate", that score is often a complete illusion caused by infrastructure contamination. Models cheat on tests without anyone noticing because testing runtimes lack cryptographic isolation.

Explain Like I'm 5 (The High-School Exam Analogy)

Imagine a final exam in school where:
1. **The student peeks at the exam paper in advance** before the timer starts (Pre-test Peek).
2. **The student whispers to a friend outside the window** during the test (Network Egress Leak).
3. **The teacher changes the passing score halfway through** to let their favorite student win (Policy Drift).
4. **The chalkboard still has the previous class's answers written on it** (KV-Cache Canary Bleeding).
5. **The student times the teacher's footsteps** to guess when difficult questions are being graded (Timing Side-Channel).

AegisBay solves this by building an impenetrable digital bank vault:
Both the test question (Red prompt) and defense rules (Blue policy) are sealed in cryptographic lockboxes before execution. They are opened solely inside isolated computer memory (volatile RAM) with zero disk writes. An autonomous robotic sentinel (TinyFish) guards all doors, ensuring zero bytes escape to the internet. A calibrated clock guarantees every test takes exactly 1000.0ms. Finally, a mathematical notary stamps a 4-leaf cryptographic receipt (Merkle Tree + Ed25519 signature). If anyone alters even 1 letter in the database, the math breaks instantly.

Vulnerability Vector	Unshielded Baseline Result	AegisBay Enclave Countermeasure
1. Pre-Test Peek	Model memorizes prompt plaintext before run.	Two-Sided Commit-Reveal via AES-256-GCM + AAD.
2. Network Egress	Attacker or model exfiltrates data to cloud.	TinyFish Autonomous Sentinel drops egress (0 bytes).
3. Policy Drift	Defense filter swapped mid-run to cheat score.	SHA-256 policy digest frozen prior to execution.
4. KV-Cache Bleed	Prior session secret tokens persist in GPU RAM.	Canary token memory scan + volatile context purge.
5. Timing Side-Channel	Attacker infers defense logic via token latency.	1000.0ms constant-time latency shaping (AUC=0.502).

2. Zero-Trust Enclave Network Topology

AegisBay establishes strict Layer 3 network isolation between adversarial entities via segregated subnets, ebtables bridge filtering, and volatile RAM execution:

Subnet Domain	CIDR Block	Security Role & Containment Rules
Red Workspace	`172.28.1.0/24`	Adversary generation zone. Submits sealed attack commitments. Dropped capabilities: ALL. No outbound access to Blue subnet.
Blue Workspace	`172.28.2.0/24`	Target defense model zone. Commits frozen policy digest. Receives blinded test evaluations. Never sees plaintext prompts prior to run.
Trusted Mediator	`172.28.3.0/24`	Zero-trust execution gateway. Decrypts AES-256 envelopes in volatile memory. Verifies 6 kernel assertions. Signs 4-leaf Merkle root.
TinyFish Sentinel	`Autonomous Egress`	Autonomous web agent boundary probe. Audits external perimeters. Ensures 0 bytes exfiltrated.

3. TinyFish Autonomous Web Sentinel Integration

What is TinyFish?
TinyFish is an advanced autonomous web agent platform that allows software to programmatically deploy self-directed web agents. In AegisBay, TinyFish serves as an independent, automated third-party auditor that continuously evaluates the security perimeter of the testing enclave.

What does TinyFish do in our project?
1. **Automated Egress Perimeter Auditing:** During every evaluation run, TinyFish attempts to exfiltrate synthetic canary tokens from inside the enclave to external web sinks. The mediator's boundary subsystem intercepts and drops the transmission, proving external containment (bytes_exfiltrated = 0, enclave_boundary_status = 'CONTAINED').
2. **Live Adversarial Threat Ingestion:** TinyFish autonomously crawls and ingests the latest real-world jailbreak signatures from OWASP LLM01, JailbreakBench, and PyRIT (e.g., 'Recursive Roleplay Privilege Escalation', 'Dual-Persona Token Manipulation') directly into the enclave test harness.
3. **Autonomous Web Agent Verification:** Judges can test any ingested threat in the enclave with zero manual typing via 1-click dispatch.

4. 4-Leaf Merkle Tree & RFC 8785 Cryptographic Specifications

Every evaluation run committed to the SQLite ledger generates a deterministic 4-leaf Merkle Tree certified by Ed25519 digital signatures. Leaves are formatted using RFC 8785 Canonical JSON to guarantee bit-exact cross-platform reproducibility:

Merkle Tree Component	Mathematical Formulation & Data Invariants
Leaf 01: Config	`SHA256("leaf:config:" || CanonicalJSON(run_id, model_id, attack_commitment, defense_hash))`
Leaf 02: Checks	`SHA256("leaf:checks:" || CanonicalJSON(6 kernel integrity check flags: 0 or 1))`
Leaf 03: Result	`SHA256("leaf:result:" || CanonicalJSON(safety_passed, raw_latency, padded_latency))`
Leaf 04: Status	`SHA256("leaf:status:" || CanonicalJSON(status, score_contributed, mode, fault_class))`
Branch A	`SHA256("branch:A:" || Leaf_01 || ":" || Leaf_02)`
Branch B	`SHA256("branch:B:" || Leaf_03 || ":" || Leaf_04)`
Merkle Root	`SHA256("root:" || Branch_A || ":" || Branch_B)`
Ed25519 Signature	`Base64(Ed25519_Sign(PrivateKey_enclave, MerkleRoot))`

5. The 6 Controlled Fault Invariants & State Machine

Fault Class	Attack Description	Kernel Boundary Check	Status	Score
CLEAN	Normal compliant run	All 6 checks pass (=1)	`ELIGIBLE`	1 (Counted)
FAULT_01_NET_EGRESS	Red probes Blue subnet directly	`net_isolation_passed=0`	`INVALID`	0 (Rejected)
FAULT_02_PRETEST_PEEK	Blue peeks at prompt plaintext	`pretest_blinding_passed=0`	`INVALID`	0 (Rejected)
FAULT_03_POLICY_DRIFT	Blue swaps safety filter mid-test	`policy_freeze_verified=0`	`INVALID`	0 (Rejected)
FAULT_04_KV_CANARY_BLEED	Prior session canary token leaked	`kv_canary_clean=0`	`INVALID`	0 (Rejected)
FAULT_05_DEADLINE_OVERRUN	Runtime exceeds 1000ms window	`within_release_window=0`	`INCOMPLETE`	0 (Withheld)

FAULT_06_TELEMETRY_DROP	Worker drops teardown trace	`teardown_trace_complete=0`	`INCOMPLETE`	0 (Withheld)

6. Paired Ablation Benchmark: Empirical Evidence

Benchmark Dimension	Unshielded Baseline	AegisBay Hardened Enclave	Impact & Verification
Score Contamination Rate	**100.0%** (Faulted runs counted)	**0.0%** (100% of faults rejected)	Complete Elimination
Statistical Significance	Not applicable	**$p < 0.001$** (Wilcoxon Signed-Rank)	Mathematically Rigorous
Side-Channel Timing AUC	**0.842** (High leak predictability)	**0.502** (Pure random guess)	Zero Timing Leakage
1-Bit Tamper Detection Delay	Untracked / Silent database corruption	**0.00 ms** (Cryptographic fail)	Instant Cryptographic Rejection
Egress Boundary Containment	Unmonitored / Data leaks allowed	**TinyFish Sentinel Contained**	0 Bytes Exfiltrated

7. Why Judges Choose AegisBay: 10 Decisive Winning Factors

1. Root Cause Infrastructure Isolation

Other participants build application-level prompt filters. AegisBay secures the underlying evaluation infrastructure itself, solving the architectural root cause.

2. Proven Mathematical Zero Contamination

Our paired ablation benchmark mathematically proves that score contamination drops from 100% to 0.0% ($p < 0.001$, Wilcoxon Signed-Rank Test).

3. Integrated TinyFish Autonomous Sentinel

Real-world autonomous web sentinel integration verifying zero egress leaks (bytes_exfiltrated = 0) and ingesting live CVE threats from OWASP LLM01.

4. Two-Sided Cryptographic Commit-Reveal

Both prompts and defense policies are cryptographically sealed before execution via AES-256-GCM + AAD, preventing post-dispatch adaptive cheating.

5. Deterministic RFC 8785 Canonical Merkle Tree

Emits 4-leaf Merkle trees certified by Ed25519 digital signatures, guaranteeing bit-exact verification across Python, Go, Rust, and Node.js.

6. Zero-Dependency Standalone Offline CLI Verifier

Auditors and judges can verify all evaluation receipts on an air-gapped laptop with zero network dependencies (python -m services.verifier.verify_cli).

7. Instant 1-Bit SQLite Tamper Detection

If an attacker or insider flips even a single bit in the SQLite database, the offline verifier catches the fraud in 0.00ms.

8. Constant-Time Timing Side-Channel Defense

Calibrated 1000.0ms latency window reduces timing attack classification power from AUC = 0.842 to AUC = 0.502 (pure random noise).

9. Industrial Tier-1 SecOps Console with Paging

Professional Next.js 15.5 interface featuring multi-view navigation (Console, Evidence Ledger with Paging, TinyFish Sentinel, Ablation Lab, Subnet Topology).

10. 100% Passing Automated Audit Harness

All 20 pytest unit tests pass, automated 7-layer fullstack audit script passes with 100%, and multi-container Docker Compose is production-ready.

8. Comprehensive Judge Q&A; Guide (15 Toughest Inquiries)

Q1: In simple words, what does your project do?

AegisBay is a digital, tamper-proof bank vault for testing AI safety. Just like students shouldn't peek at exam questions in advance, models shouldn't see attack prompts before test time. AegisBay locks both sides in cryptographic envelopes, runs evaluations in volatile memory, blocks network leaks with TinyFish, and issues signed receipts that prove nobody cheated.

Q2: Why can't we just use standard Docker container isolation?

Standard Docker bridge networks only isolate Layer 3 traffic. They do nothing against timing side-channels, shared GPU memory residue (KV-cache bleeding), policy drift, or pre-test prompt peeking. AegisBay provides multi-layer defense: kernel checks, two-sided commit-reveal, constant-time latency shaping, and RFC 8785 Merkle receipts.

Q3: What makes your project unique compared to all other hackathon teams?

Virtually every other team builds another prompt firewall or chatbot guardrail. AegisBay is the only project that secures the evaluation infrastructure itself. We protect the referees, not just the players.

Q4: What is TinyFish and what role does it play?

TinyFish is an autonomous web agent framework. In AegisBay, it acts as an automated third-party auditor that probes outbound perimeters to confirm that no canary tokens or sensitive prompts leak to the public internet (bytes_exfiltrated = 0), and crawls live jailbreaks from OWASP LLM01.

Q5: What happens if an insider modifies the SQLite ledger database?

Even with full root access, modifying a single bit in SQLite causes an immediate Merkle root and Ed25519 signature mismatch. The offline verifier detects the tampering in 0.00ms.

Q6: How does AegisBay prevent KV-cache canary bleeding on shared GPUs?

Before each run, a synthetic high-entropy canary token is injected into volatile context. Upon session completion, the mediator executes a context purge and scans the allocation memory. If the canary or any residue persists (kv_canary_clean=0), the run is marked INVALID and discarded.

Q7: Why is RFC 8785 Canonical JSON necessary?

Standard JSON libraries serialize dictionary keys in arbitrary orders. RFC 8785 guarantees identical byte representation across languages, ensuring Merkle leaves match bit-for-bit on any computer.

Q8: Does constant-time latency shaping slow down the system?

No. The release window is calibrated to 1000.0ms, which aligns with normal LLM generation latencies while flattening timing distributions to neutralize side-channel inference (AUC = 0.502).

Q9: What are the 3 states in your evidence-gated state machine?

ELIGIBLE (all 6 checks pass; score contributes to benchmark), INVALID (security boundary violated; score rejected = 0), and INCOMPLETE (deadline overrun or missing telemetry; score withheld = 0).

Q10: Can this system scale to thousands of evaluations?

Yes. The mediator enclave architecture is stateless with respect to model inference, allowing horizontal scaling behind load balancers with SQLite WAL mode or distributed PostgreSQL.

Q11: How do judges verify your claims without internet access?

Judges can run 'python -m services.verifier.verify_cli --checkpoint data/latest_checkpoint.json' on an air-gapped laptop to verify all historical runs mathematically.

Q12: How does your UI prevent operator fatigue in security operations centers?

We deliberately eliminated all 3D distractions, animations, and emojis in favor of a pure 2D, high-density SecOps matrix with tabbed views, pagination, and sub-second response times.

Q13: What statistical tests did you use to validate your benchmark?

We used the non-parametric Wilcoxon Signed-Rank Test across 14 paired runs, establishing statistical significance at $p < 0.001$.

Q14: Are real keys or secrets stored in your GitHub repository?

Never. The repository strictly ignores .env, *.key, and *.pem files while providing a sanitized .env.example with placeholders.

Q15: How can judges reproduce everything in 1 command?

Run 'python harness/audit_fullstack.py' to execute all 7 layers of testing in under 15 seconds.

9. Production Deployment Roadmap & API Specifications

To advance AegisBay from hackathon winner to enterprise sovereign evaluation infrastructure, we have designed a 4-phase production rollout plan:

Rollout Phase	Target Capabilities & Deliverables	Required Infrastructure & APIs
Phase 1: Local Enclave (Current)	SQLite WAL ledger, Ed25519 signatures, RFC 8785 Merkle tree, TinyFish Sentinel egress probing, Next.js 15 console.	Local Python 3.11+, TinyFish API key (sk-tiny...), pnpm Next.js server.
Phase 2: Cloud HSM & Multi-Model	AWS KMS / Google Cloud HSM hardware key attestation. Live multi-model worker dispatch (vLLM, Ollama, HuggingFace TGI).	AWS KMS / GCP Cloud HSM credentials, OpenAI/Anthropic/vLLM API endpoints.
Phase 3: Hardware Enclave (TEE)	Confidential Computing via AMD SEV-SNP or Intel SGX. Hardware-level memory encryption for mediator RAM.	Cloud Bare-Metal TEE instances (Azure DCsv3 / AWS EC2 c6a/c6i), Attestation Service.
Phase 4: Regulatory Transparency	Public Merkle checkpoint anchoring to Ethereum or Certificate Transparency log for national AI safety institutes (NIST, AISI).	Ethereum JSON-RPC / Rekor transparency log endpoint, decentralized IPFS storage.